

| Gerenciamento De Memória

João Eduardo Montandon

DCC024

| Introdução

Registros de Ativação, Objetos, `new`, `malloc`, etc...

O que tem em comum? 🤔

- Todos precisam ser alocados, recuperados, e desalocados de alguma forma.
 - Quem? *Gerenciador de memória em tempo de execução*
 - Como? É o que veremos...
-

| Modelo De Memória

- Por enquanto, assuma que o SO forneça blocos de memória de tamanho fixo para os programas que serão executados.
 - Modelar essas regiões como vetores em Java.
-

```
public class MemoryManager {  
    private int[] memory;  
  
    /**  
     * Método construtor.
```

```
*  
* @param initialMemory região de memória  
* disponibilizada pelo SO.  
*/  
public MemoryManager(int[] initialMemory) {  
    this.memory = initialMemory;  
}  
}
```

| Tipos De Memória

Onde essas coisas são armazenadas em um programa? 🤔

| Modos De Armazenamento

1. Memória estática
2. Memória de pilha
3. Memória de heap

| Memória Estática

- Reservada para dados declarados uma vez, mas persistidos durante todo o programa.
- Alocação e desalocação acontecem uma única vez!

```
#include <stdio.h>  
void counter1() {  
    static store = 0;
```

```
    store++;  
    printf("Counter 1 = %d\n", store);  
}  
void counter2() {  
    static store = 0;  
    store++;  
    printf("Counter 2 = %d\n", store);  
}  
int main() {  
    counter1();  
    counter2();  
    counter2();  
    counter2();  
    counter1();  
}
```

O que vai ser impresso nesse programa? 🤔

Por que?

| Pilha

- Reservada para dados locais, manipulados dentro de chamadas de funções.
- Registro de Ativação (RA) da função é alocado quando função é chamada.
 - Contém informações locais da função.
- RA é desalocado quando função é finalizada.
- Comportamento LIFO: Último RA a entrar, é o primeiro a sair.

Por que pilha? 🤔

| Funcionamento

1. `push(int size)` : Alocará um RA de tamanho `size` .
2. `pop()` : Desalocará o RA presente no topo da pilha.

3. **top**: Endereço do RA atual (em execução).
4. Cada alocação reservará espaço do RA + Posição do topo anterior.

top: 8 →

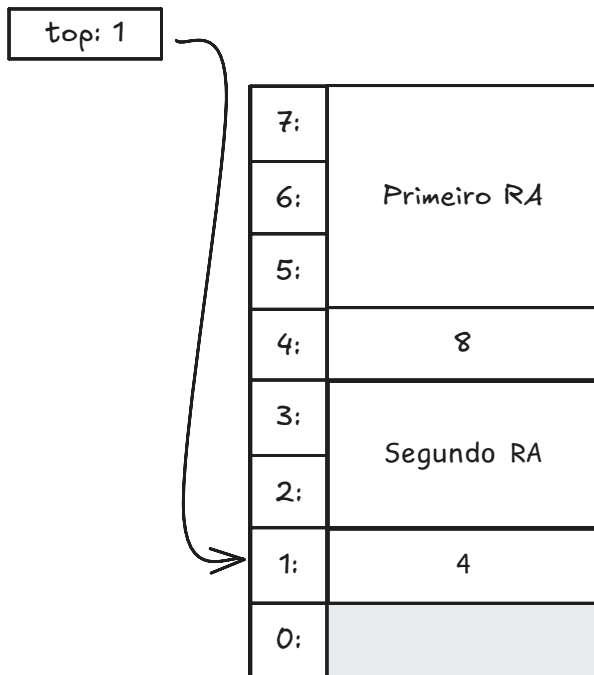
7:	
6:	
5:	
4:	
3:	
2:	
1:	
0:	

```
int[] arr = new int[8];  
MemoryManager m = new MemoryManager(arr);
```

top: 4

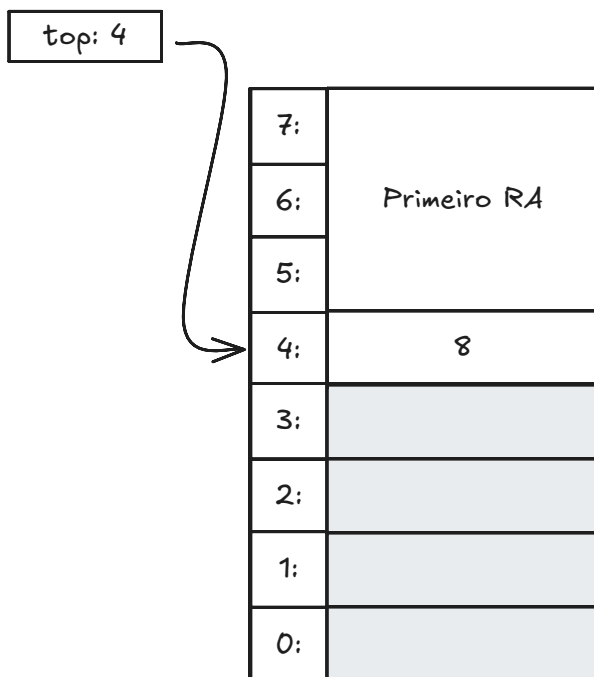
7:	Primeiro RA
6:	
5:	
4:	8
3:	
2:	
1:	
0:	

```
int[] arr = new int[8];  
MemoryManager m = new MemoryManager(arr);  
  
m.push(3);
```



```
int[] arr = new int[8];
MemoryManager m = new MemoryManager(arr);

m.push(3);
m.push(2);
```



```
int[] arr = new int[8];
MemoryManager m = new MemoryManager(arr);

m.push(3);
m.push(2);

m.pop();
```

Implementação

```
import java.lang.StackOverflowError;

public class MemoryManager {
```

```

    private int[] memory;
private int top;

    /**
     * Método construtor.
     *
     * @param initialMemory região de memória
     * disponibilizada pelo SO.
     */
    public MemoryManager(int[] initialMemory) {
        this.memory = initialMemory;
        this.top = this.memory.length;
    }

    /**
     * Aloca um bloco de memória e retorna seu endereço
     *
     * @param memorySize tamanho do bloco a ser alocado
     * @return endereço do bloco alocado
     * @throws StackOverflowError se alocação ultrapassar
     * tamanho da pilha
     */
    public int push(int memorySize) {
        int size = memorySize + 1;
        int newTop = top;

        newTop -= size;
        if (newTop < 0)
            throw new StackOverflowError();

        memory[newTop] = this.top;
        this.top = newTop;

        return this.top + 1;
    }

    /**
     * Desempilha bloco do topo da pilha.
     * Funciona apenas se pilha não estiver vazia.

```

```
*/  
public void pop() {  
    this.top = memory[this.top];  
}  
  
public static void main(String[] args) {  
    int[] arr = new int[8];  
    MemoryManager m = new MemoryManager(arr);  
  
    System.out.println(m.push(3));  
    System.out.println(m.push(2));  
    m.pop();  
}  
}
```

| Heaps

- Pilha: pressupõe uma ordem entre alocação e desalocação.
- E se os dados não puderem seguir essa ordem? 🤔

Um **heap** é um conjunto de blocos de memória, com uma interface que permite alocação e desalocação sem ordem pré-definida.

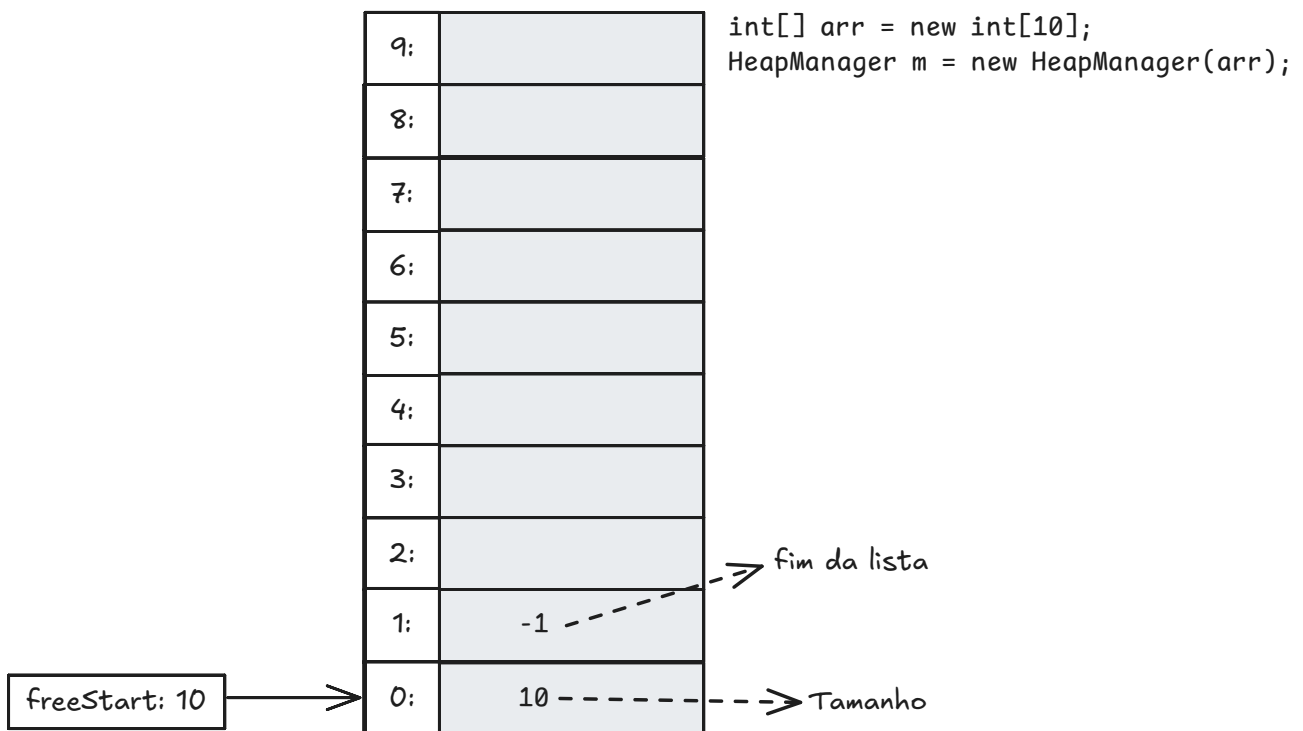
Não confundir com a estrutura de dados!

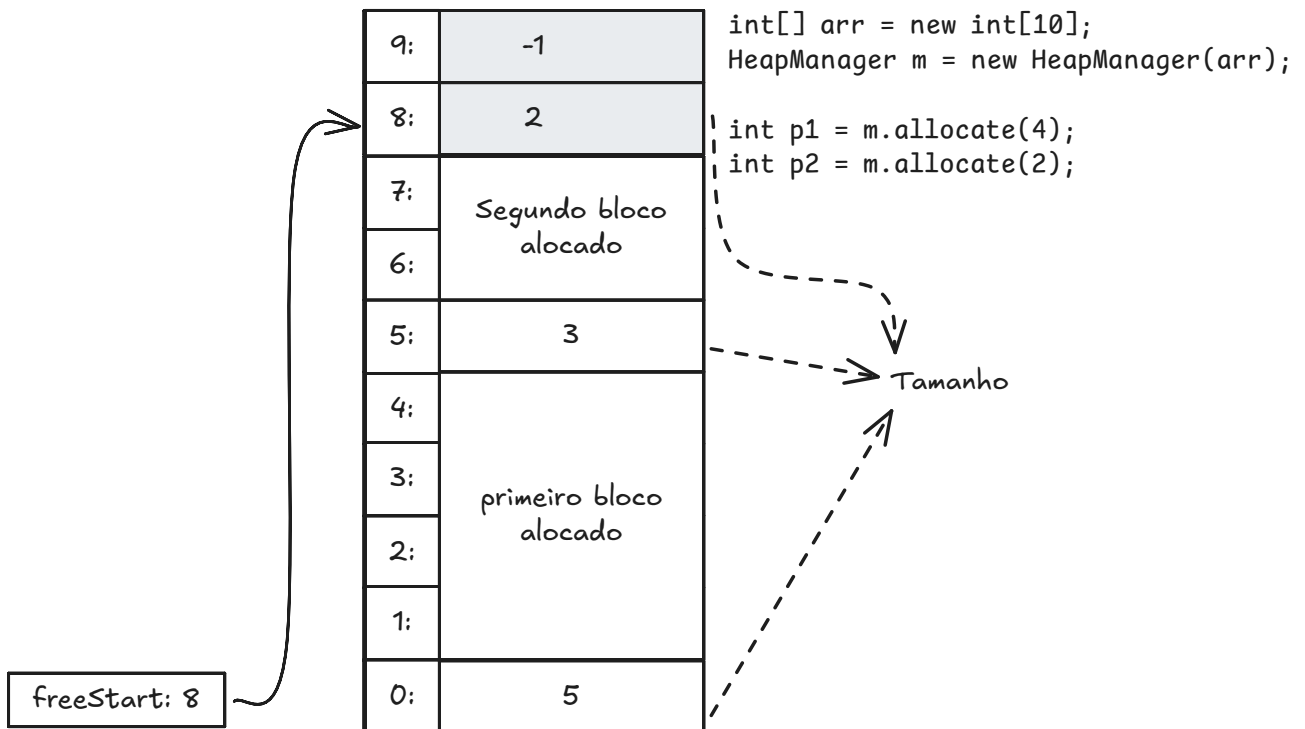
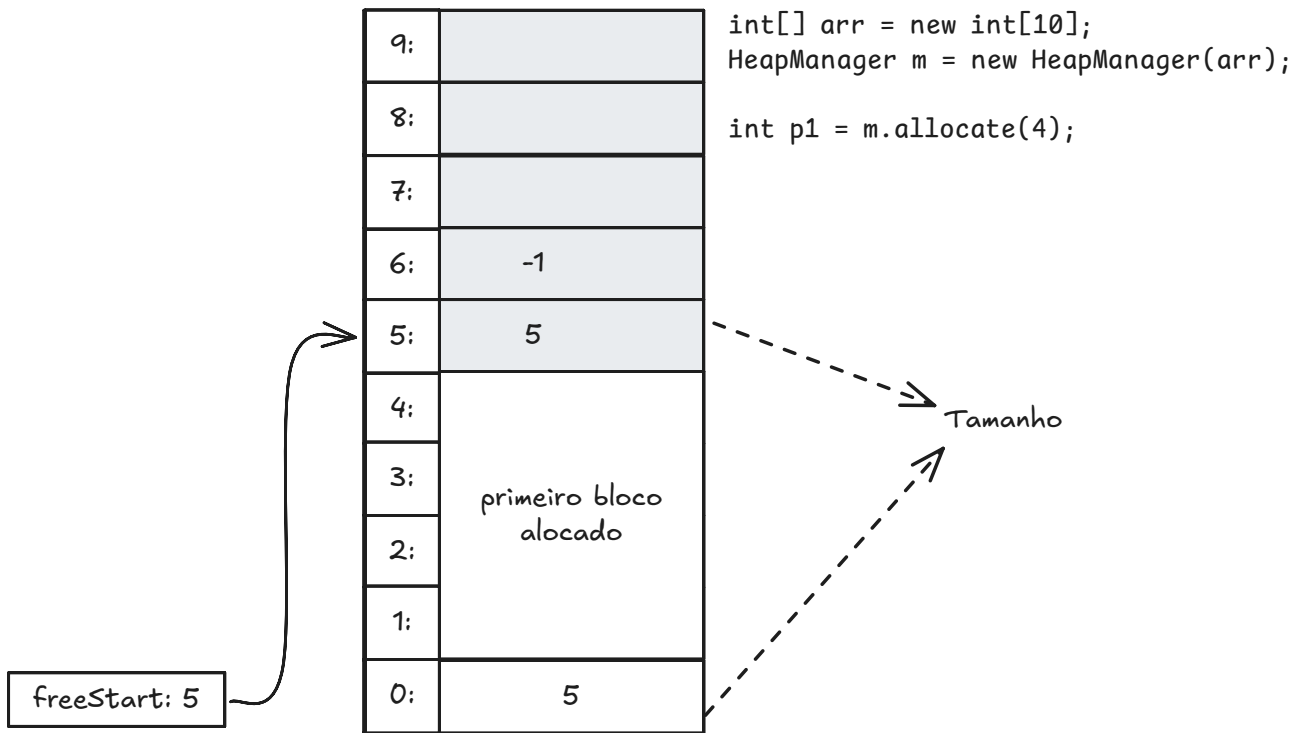
| First-Fit

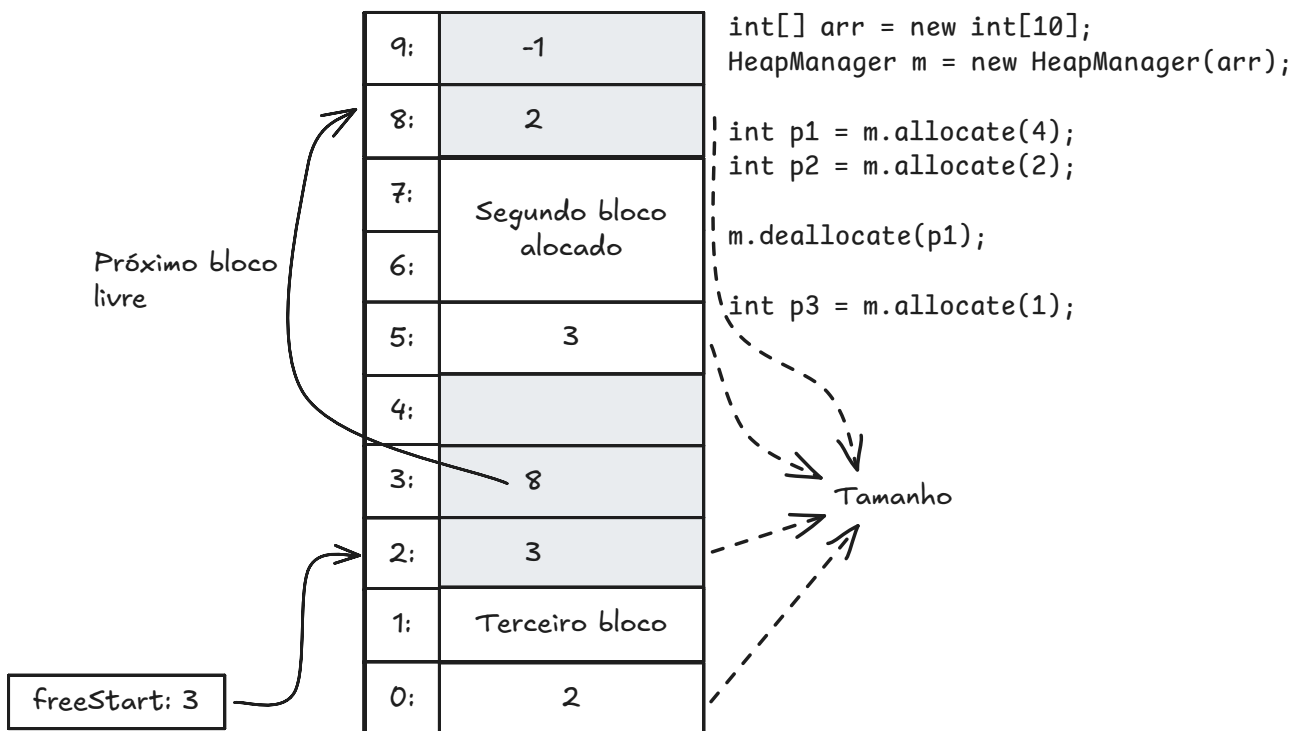
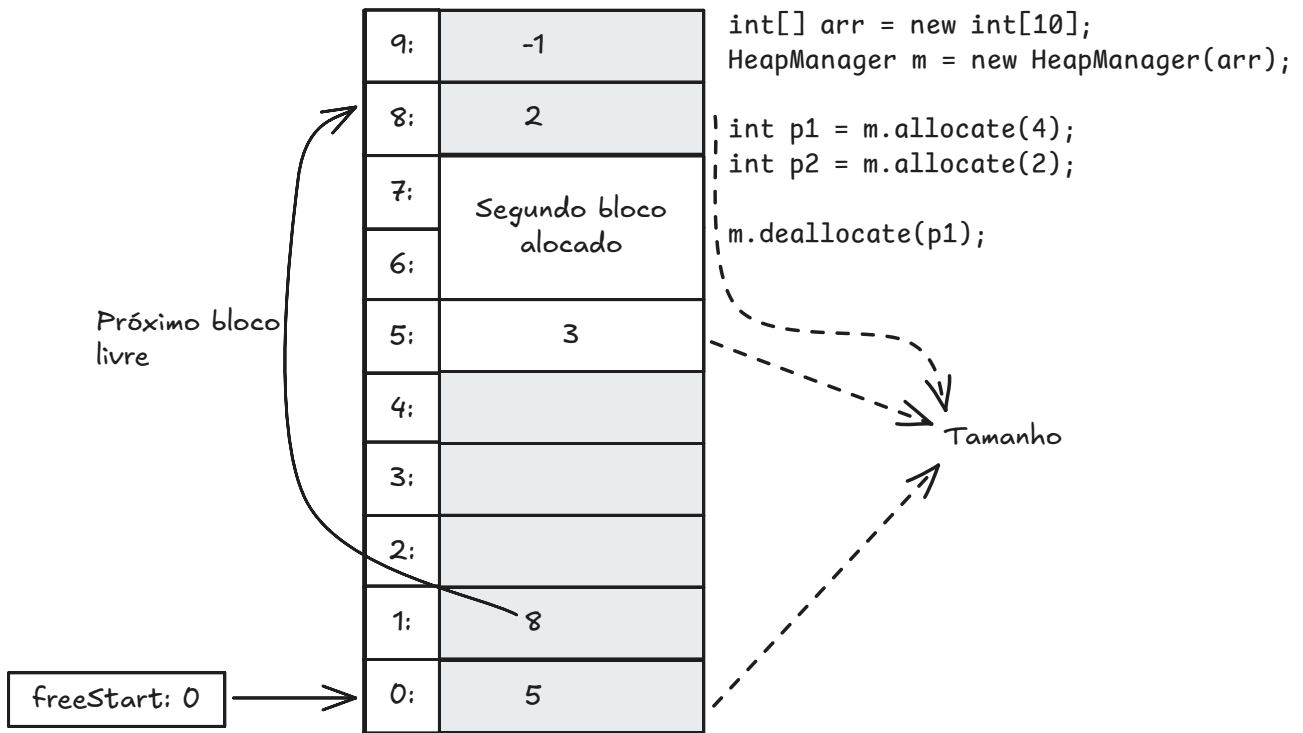
1. Lista encadeada de blocos livres, começando com um grande bloco.
2. `allocate(int n)`: Aloca um bloco de tamanho `n` no primeiro espaço livre que encontrar.

- Se espaço livre for maior que `n`, reserva no início do bloco e retorna o restante.
3. `deallocate(int pos)`: Libera bloco iniciado em `pos` incluindo-o na lista de blocos livres.

- `freeStart`: cabeça da lista de blocos livres.
- Todo bloco, *alocado ou livre*, tem seu tamanho alocado na primeira palavra.
- Blocos livres possuem o link para o próximo bloco livre OU -1 se for fim da lista.







Implementação

```

public class HeapManager {

    private static final int NULL = -1;
    private int[] memory;
    private int freeStart;

    /**
     * Construtor do Heap
     * @param initialMemory memória a ser gerenciada pelo
heap.
     */
    public HeapManager(int[] initialMemory) {
        this.memory = initialMemory;
        this.memory[0] = memory.length;
        this.memory[1] = NULL;
        freeStart = 0;
    }

    /**
     * Aloca um bloco e retorna seu endereço.
     *
     * @param n Tamanho do bloco a ser alocado
     * @return endereço do bloco alocado
     * @throws OutOfMemoryError se não há bloco grande o
     * suficiente para alocar espaço.
     */
    public int allocate(int n) {
        // tamanho real a ser alocado
        int size = n + 1;

        // busca primeiro bloco para ser alocado
        int p = this.freeStart;
        int previousP = NULL;
        while (p != NULL && this.memory[p] < size) {
            previousP = p;
            p = this.memory[p+1];
        }

        // se não encontrar, lança exceção
    }
}

```

```

        if (p == NULL)
            throw new OutOfMemoryError();

        int nextFree = this.memory[p+1];

        // Se bloco possuir tamanho maior do que o
necessário,
        // ocupa o início do bloco e retorna o restante
como
        // parte liberada da memória.
        int remaining = this.memory[p] - size;
        if (remaining > 1) {
            // ocupa a primeira parte do bloco
            this.memory[p] = size;

            // preenche dados do bloco livre restante
            nextFree = p + size;
            this.memory[nextFree] = remaining;
            this.memory[nextFree+1] = this.memory[p+1];
        }

        // Atualiza próximo bloco livre da lista
        if (previousP == NULL)
            this.freeStart = nextFree;
        else
            this.memory[previousP + 1] = nextFree;

        return p+1;
    }

/**
 * Desaloca um bloco de memória do heap.
 *
 * @param contentAddress endereço do conteúdo
 * armazenado no heap.
 */
public void deallocate(int contentAddress) {
    int blockAddress = contentAddress - 1;
    this.memory[contentAddress] = freeStart;

```

```
        this.freeStart = blockAddress;
    }

    public static void main(String[] args) {
        int[] arr = new int[10];
        HeapManager m = new HeapManager(arr);

        int p1 = m.allocate(4);
        System.out.println("p1=" + p1);

        int p2 = m.allocate(2);
        System.out.println("p2=" + p2);

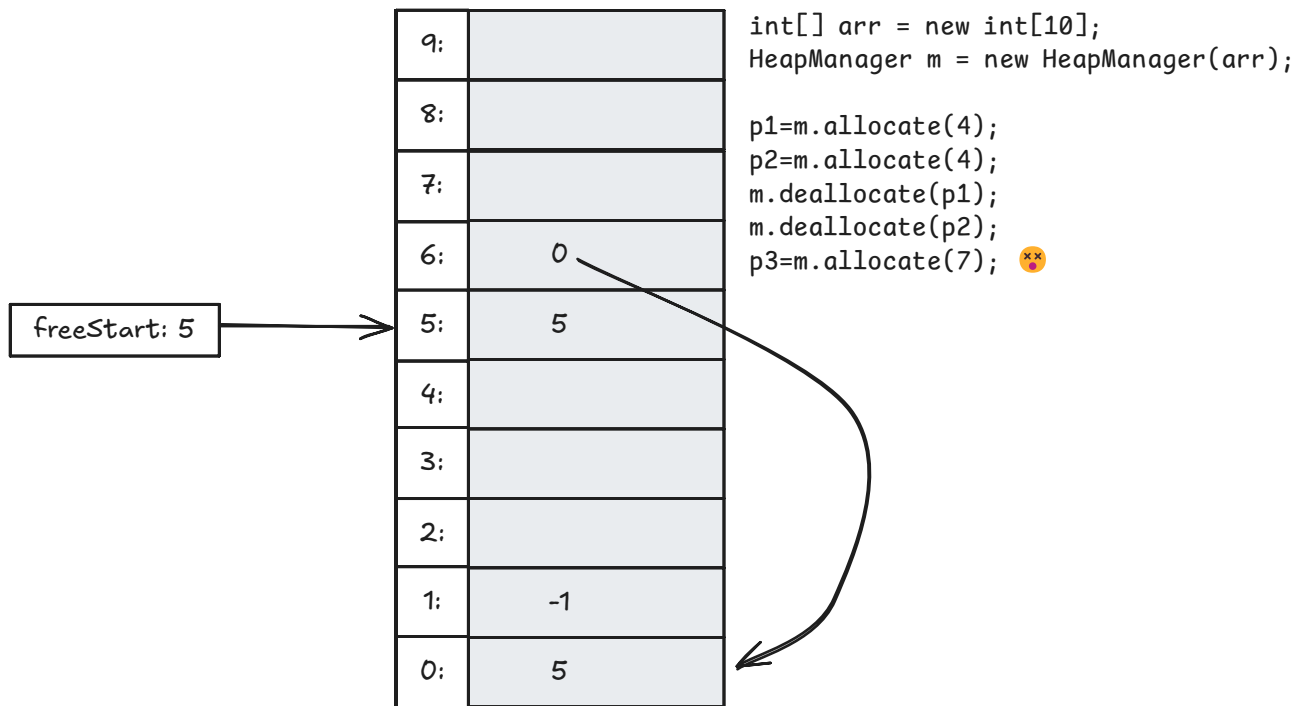
        m.deallocate(p1);

        int p3 = m.allocate(1);
        System.out.println("p3=" + p3);
    }
}
```

Fusão De Blocos

```
p1=m.allocate(4);
p2=m.allocate(4);
m.deallocate(p1);
m.deallocate(p2);
p3=m.allocate(7);
```

Isso funciona? 🤔



O que aconteceu?

- Temos dois blocos de tamanho 5, não um bloco de tamanho 10!
- **Solução:** fundir blocos livres adjacentes.

deallocate v2

- Ao liberar um bloco, verificar os adjacentes.
- Se adjacente também for livre, fundir.

```
public void deallocate(int contentAddress) {
    int blockAddress = contentAddress - 1;

    // Realiza busca para verificar se existe bloco
    // livre antes ou depois do bloco a ser
    desalocado
    int p = this.freeStart;
```

```

int previousP = NULL;
while(p ≠ NULL && p < blockAddress) {
    previousP = p;
    p = this.memory[p+1];
}

// se adjacente estiver logo após, faz
// a fusão entre os dois blocos
if(blockAddress + this.memory[blockAddress] == p)
{
    this.memory[blockAddress] += this.memory[p];
    p = this.memory[p+1];
}

// Nosso bloco será liberado primeiro
if(previousP == NULL) {
    this.freeStart = blockAddress;
    this.memory[blockAddress+1] = p;

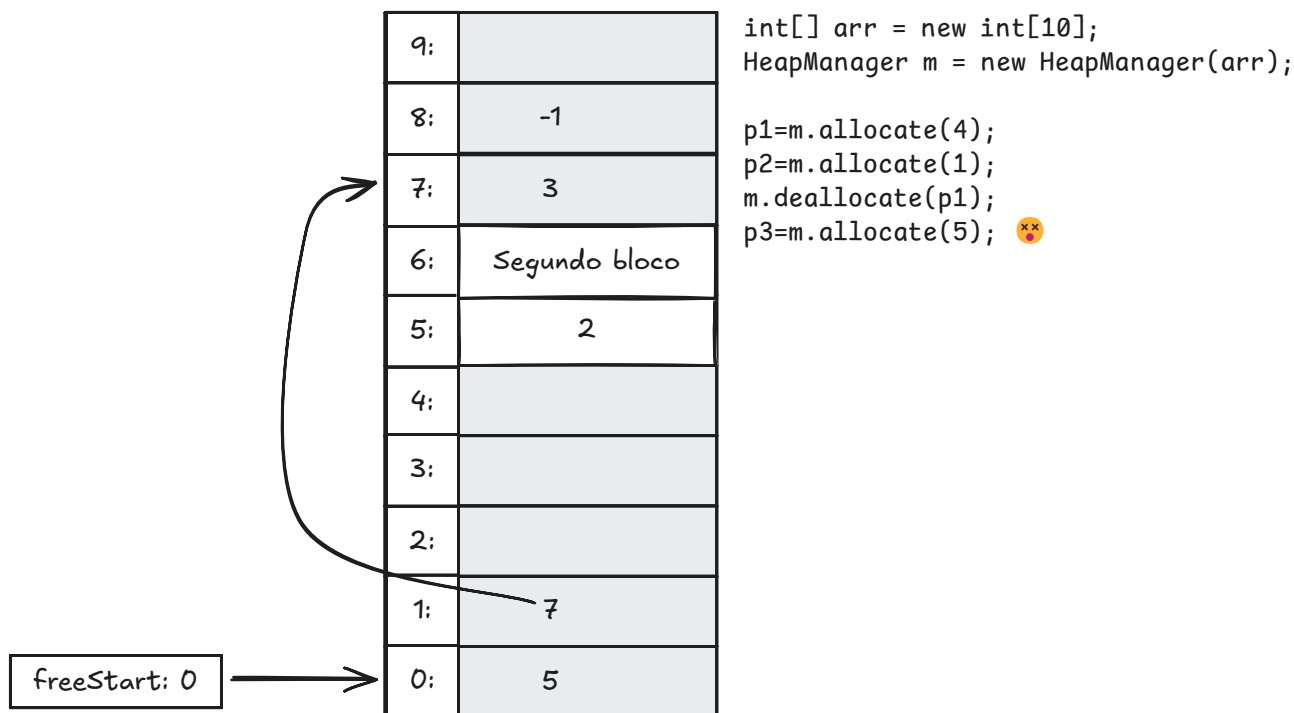
    // Se bloco anterior é livre, fazer a fusão
} else if (previousP + this.memory[previousP] ==
blockAddress) {
    this.memory[previousP] +=
memory[blockAddress];

    // Não há fusão a ser feita
} else {
    this.memory[previousP+1] = blockAddress;
    this.memory[blockAddress+1] = p;
}
}

```

| Fragmentação

E se os blocos livres estiverem separados por um ocupado? 🤔



| Gerenciamento Do Heap

Três problemas principais

1. Localização: onde alocar um bloco?
2. Divisão: quando e como dividir um bloco?
3. Fusão: Quando e como recombinar blocos?

1. Localização: **first-fit**, best-fit, next-fit
 - Estruturas alternativas: **listas**, árvores, tabelas hash, etc.
2. Divisão: **Dividir conforme tamanho do bloco**, tamanho-padrão, etc.
3. Fusão: Sem, **eager**, delayed.

| Problemas Comuns

Quais problemas mais comuns ao lidar com variáveis alocadas no heap?

```
#include <stdio.h>
#include <stdlib.h>
void problem() {
    int* i = (int*) malloc (sizeof(int));
    *i = 3;
    printf("%d\n", *i);
}
int main() {
    problem();
}
```

Qual o problema aqui? 😬

E aqui? 😬

```
#include <stdio.h>
#include <stdlib.h>
void dangling() {
    int* i = (int*) malloc (sizeof(int));
    int* j;
    *i = 3;
    free(i);
    j = (int*) malloc (sizeof(int));
    *j = 8;
    printf("%d\n", *i);
}
int main() {
    dangling();
}
```

- Já que muitos erros são causados pelo mal gerenciamento da memória... 😬

- ... E isso muitas das vezes é um fardo para o desenvolvedor... 🤔
 - ...
 - ... Por que não delegar isso para a linguagem?
-

| Coletor De Lixo

Processo que, periodicamente, busca por blocos não utilizados, e os libera para uso.

1. Mark and Sweep
 2. Cópia
 3. Contador de Referências
-

| Mark and Sweep

1. Faz a busca por blocos no Heap marcando os que estão em uso.
 2. Elimina os blocos que não foram marcados.
-

| Cópia

- Gerenciador do Heap usa apenas metade do espaço disponível.
 - Quando metade está ocupada, ele vai copiar os blocos em uso para a outra metade.
 - Esse processo é repetido toda vez que uma metade ficar cheia.
-

| Contador De Referências

- Todo bloco possui um contador.

- Incrementado quando uma nova referência apontar para o bloco
 - Decrementado quando uma referência deixar de apontar para o bloco.
 - Blocos com contador iguais a zero serão liberados.
-

Muito pouco utilizado, consegue imaginar o por quê? 🤔

| Considerações Finais

- Performance e confiabilidade são fatores críticos para o gerenciamento de memória.
 - Como balancear? 🤔
- Cada técnica possui sua particularidade e melhor cenário de uso.
- Coletor de lixo: recurso muito importante em linguagens modernas.
 - Área de pesquisa muito ativa, atualmente.